Día 5

Listas

Hay cuatro tipos de datos de recolección en Python:

- Lista: es una colección que se ordena y se puede cambiar (modificar). Permite miembros duplicados.
- Tupla: es una colección que está ordenada e inmutable o inmodificable (inmutable). Permite miembros duplicados.
- Set: es una colección que no está ordenada, no indexada y no modificable, pero podemos agregar nuevos elementos al conjunto. No se permiten miembros duplicados.
- Diccionario: es una colección que no está ordenada, cambiable (modificable) e indexada. Sin miembros duplicados.

Una lista es la recopilación de diferentes tipos de datos que se ordena y modifica (mutable). Una lista puede estar vacía o puede tener diferentes elementos de tipo de datos.

Cómo crear una lista

En Python podemos crear listas de dos maneras:

- Uso de la función () integrada de la lista

```
# syntax
lst = list()
empty_list = list() # this is an empty list, no item in the list
print(len(empty_list)) # 0
```

- Usando corchetes, []

```
# syntax
lst = []
empty_list = [] # this is an empty list, no item in the list
print(len(empty_list)) # 0
```

Python 3.11
known limitations

```
1 lst = list()
2 empty_list = list() # this is an empty list, no item in the list
3 print(len(empty_list)) # 0
```

Edit this code

line that just executed
next line to execute

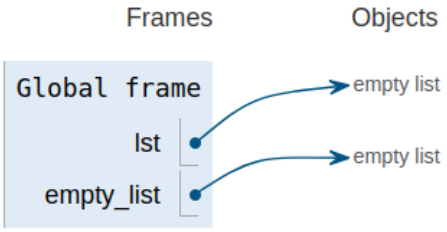
<< First | < Prev | Next > | Last >>

Done running (3 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
0
```



Listas con valores iniciales. Utilizamos *len()* para encontrar la longitud de una lista.

fruits = ['banana', 'orange', 'mango', 'lemon'] # list of fruits
vegetables = ['Tomato', 'Potato', 'Cabbage','Onion', 'Carrot'] # list of vegetables
animal_products = ['milk', 'meat', 'butter', 'yoghurt'] # list of animal products
web_techs = ['HTML', 'CSS', 'JS', 'React','Redux', 'Node', 'MongDB'] # list of web technologies
countries = ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']

```
1
2 fruits = ['banana', 'orange', 'mango', 'lemon']                    # list
3 vegetables = ['Tomato', 'Potato', 'Cabbage','Onion', 'Carrot']      # list
4 animal_products = ['milk', 'meat', 'butter', 'yoghurt']            # list
5 web_techs = ['HTML', 'CSS', 'JS', 'React','Redux', 'Node', 'MongDB'] # lis
6 countries = ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']
```

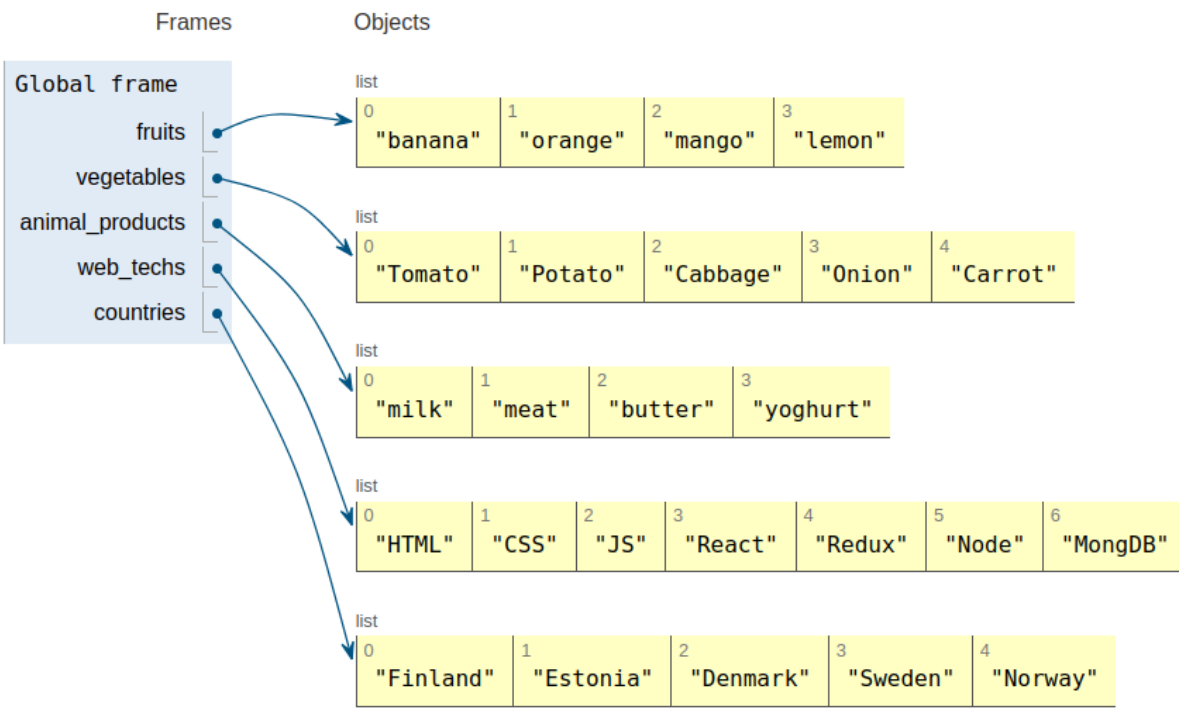
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode




```
# Print the lists and its length
print('Fruits:', fruits)
print('Number of fruits:', len(fruits))
print('Vegetables:', vegetables)
print('Number of vegetables:', len(vegetables))
print('Animal products:',animal_products)
print('Number of animal products:', len(animal_products))
print('Web technologies:', web_techs)
print('Number of web technologies:', len(web_techs))
print('Countries:', countries)
print('Number of countries:', len(countries))
```

output

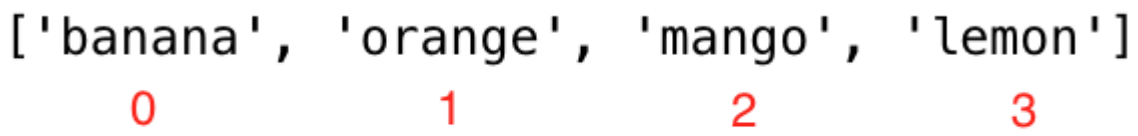
Fruits: ['banana', 'orange', 'mango', 'lemon']
Number of fruits: 4
Vegetables: ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
Number of vegetables: 5
Animal products: ['milk', 'meat', 'butter', 'yoghurt']
Number of animal products: 4
Web technologies: ['HTML', 'CSS', 'JS', 'React', 'Redux', 'Node', 'MongDB']
Number of web technologies: 7
Countries: ['Finland', 'Estonia', 'Denmark', 'Sweden', 'Norway']
Number of countries: 5

- Las listas pueden tener elementos de diferentes tipos de datos

lst = ['Asabeneh', 250, True, {'country':'Finland', 'city':'Helsinki'}] # list containing different data types

Acceso a los elementos de la lista mediante indexación positiva

Accedemos a cada elemento en una lista usando su índice. Un índice de lista comienza desde 0. La imagen de abajo muestra claramente dónde comienza el índice




```
fruits = ['banana', 'orange', 'mango', 'lemon']
first_fruit = fruits[0] # we are accessing the first item using its index
print(first_fruit)      # banana
second_fruit = fruits[1]
print(second_fruit)     # orange
last_fruit = fruits[3]
print(last_fruit) # lemon
# Last index
last_index = len(fruits) - 1
last_fruit = fruits[last_index]
```

```
1  fruits = ['banana', 'orange', 'mango', 'lemon']
2  first_fruit = fruits[0] # we are accessing the first item using its inde
3  print(first_fruit)      # banana
4  second_fruit = fruits[1]
5  print(second_fruit)     # orange
6  last_fruit = fruits[3]
7  print(last_fruit) # lemon
8  # Last index
9  last_index = len(fruits) - 1
10 last_fruit = fruits[last_index]
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (9 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
banana
orange
lemon
```

Frames | Objects

Global frame

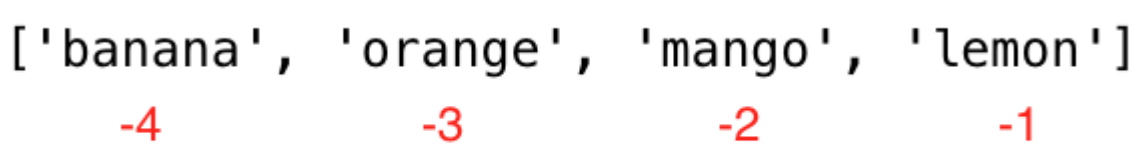
fruits	
first_fruit	"banana"
second_fruit	"orange"
last_fruit	"lemon"
last_index	3

list

0	1	2	3
"banana"	"orange"	"mango"	"lemon"

Acceso a los elementos de la lista mediante indexación negativa

La indexación negativa significa que a partir del final, -1 se refiere al último elemento, -2 se refiere al segundo último elemento.



fruits = ['banana', 'orange', 'mango', 'lemon']

```
first_fruit = fruits[-4]
last_fruit = fruits[-1]
second_last = fruits[-2]
print(first_fruit)      # banana
print(last_fruit)       # lemon
print(second_last)      # mango
```

```
1  fruits = ['banana', 'orange', 'mango', 'lemon']
2  first_fruit = fruits[-4]
3  last_fruit = fruits[-1]
4  second_last = fruits[-2]
5  print(first_fruit)      # banana
6  print(last_fruit)       # lemon
7  print(second_last)      # mango
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (7 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
banana
lemon
mango
```

Frames | Objects

Global frame

fruits	
first_fruit	"banana"
last_fruit	"lemon"
second_last	"mango"

list

0	1	2	3
"banana"	"orange"	"mango"	"lemon"

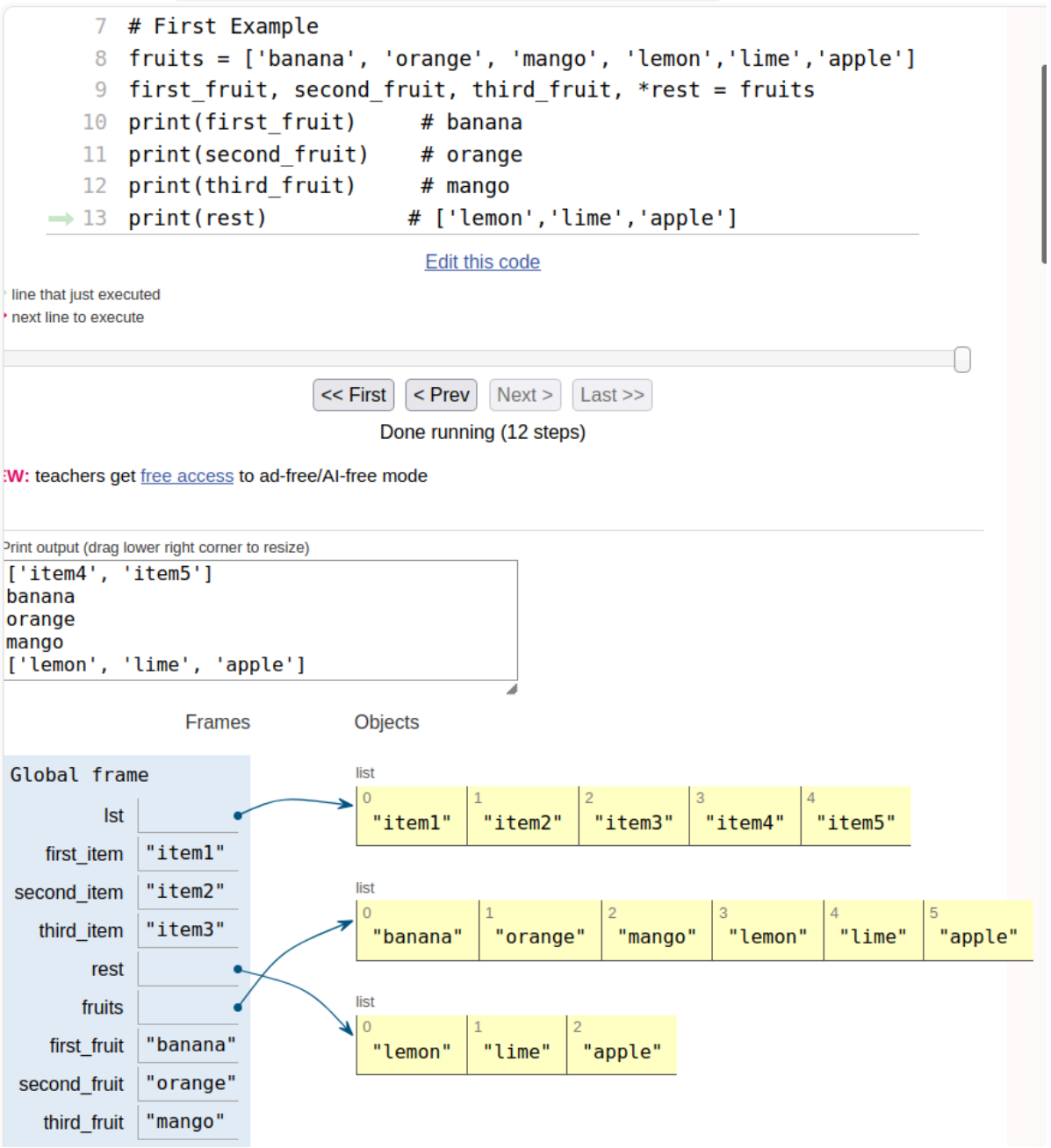
Desembalaje de artículos de lista

```
lst = ['item1','item2','item3', 'item4', 'item5']
first_item, second_item, third_item, *rest = lst
print(first_item)     # item1
print(second_item)    # item2
print(third_item)     # item3
print(rest)           # ['item4', 'item5']
```

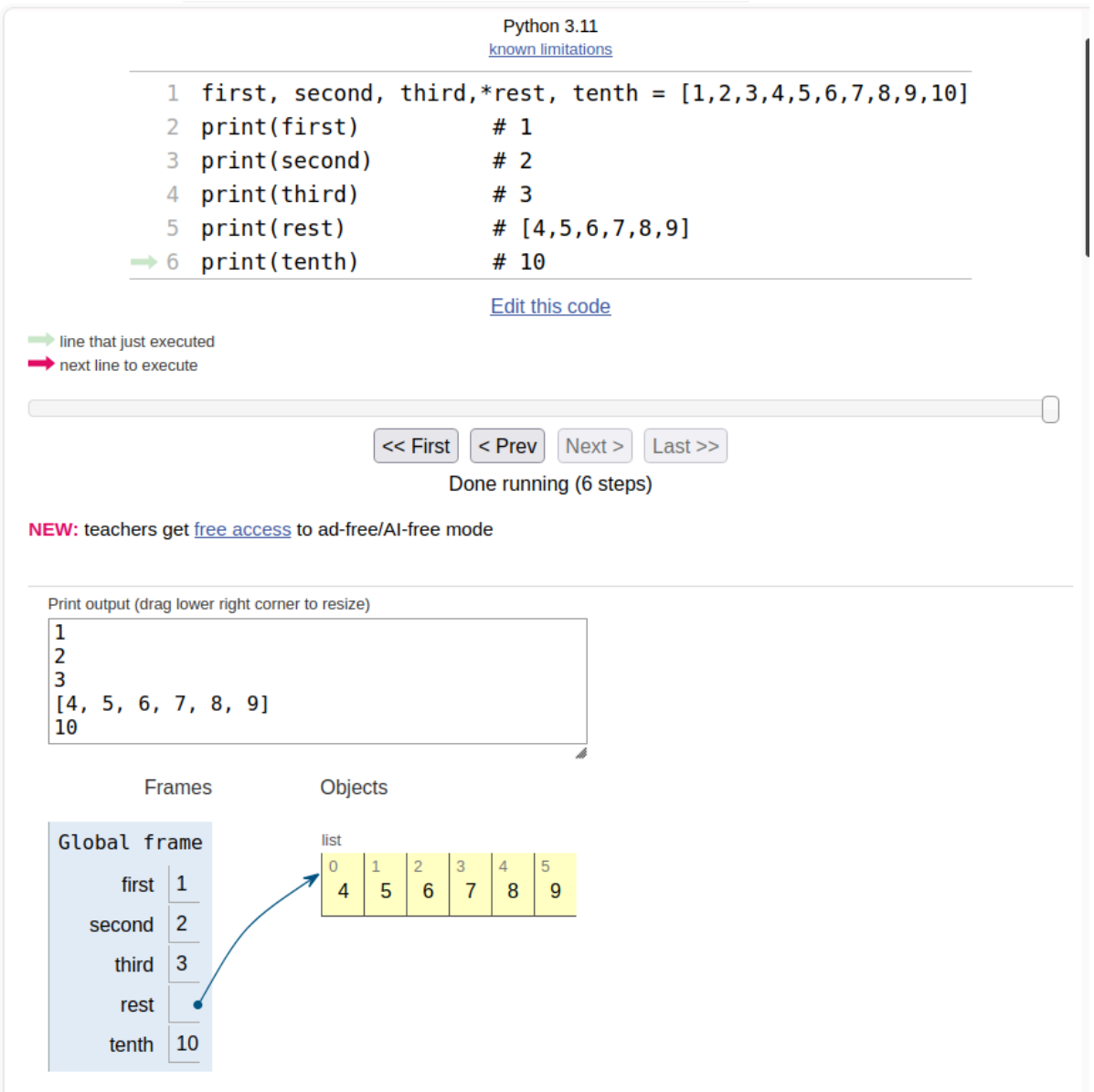
 # First Example

```
fruits = ['banana', 'orange', 'mango', 'lemon','lime','apple']
```

first_fruit, second_fruit, third_fruit, *rest = fruits
print(first_fruit) # banana
print(second_fruit) # orange
print(third_fruit) # mango
print(rest) # ['lemon','lime','apple']



 # Second Example about unpacking list
first, second, third,*rest, tenth = [1,2,3,4,5,6,7,8,9,10]
print(first) # 1
print(second) # 2
print(third) # 3
print(rest) # [4,5,6,7,8,9]
print(tenth) # 10



 # Third Example about unpacking list

```
countries = ['Germany',
'France','Belgium','Sweden','Denmark','Finland','Norway','Iceland','Estonia']
gr, fr, bg, sw, *scandic, es = countries
print(gr)
print(fr)
print(bg)
print(sw)
print(scandic)
print(es)
```

```
1 countries = ['Germany', 'France','Belgium','Sweden','Denmark','Finland','I
2 gr, fr, bg, sw, *scandic, es = countries
3 print(gr)
4 print(fr)
5 print(bg)
6 print(sw)
7 print(scandic)
8 print(es)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (8 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
France
Belgium
Sweden
['Denmark', 'Finland', 'Norway', 'Iceland']
Estonia
```

Rebanar artículos de una lista

- Indexación positiva: Podemos especificar un rango de índices positivos especificando el inicio, final y paso, el valor de retorno será una nueva lista. (valores predeterminados para inicio = 0, final = len(lst) - 1 (último elemento), paso = 1)

fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[0:4] # it returns all the fruits
# this will also give the same result as the one above
all_fruits = fruits[0:] # if we don't set where to stop it takes all the rest
orange_and_mango = fruits[1:3] # it does not include the first index
orange_mango_lemon = fruits[1:]
orange_and_lemon = fruits[::2] # here we used a 3rd argument, step. It will take every 2cnd item - ['banana', 'mango']

```
fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[0:4] # it returns all the fruits
# this will also give the same result as the one above
all_fruits = fruits[0:] # if we don't set where to stop it takes all the
orange_and_mango = fruits[1:3] # it does not include the first index
orange_mango_lemon = fruits[1:]
orange_and_lemon = fruits[::2] # here we used a 3rd argument, step. It wi
```

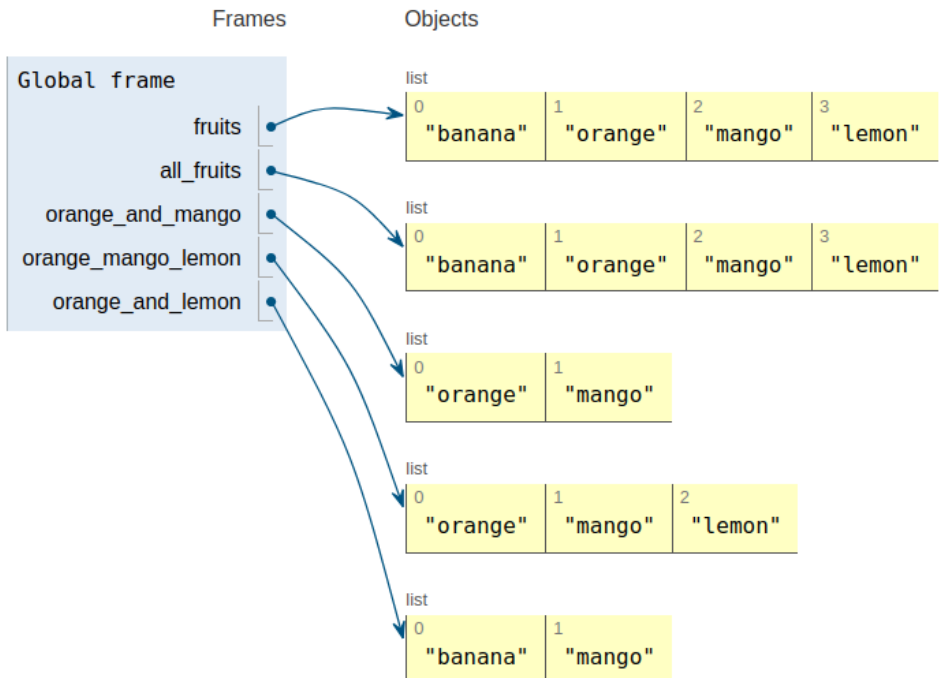
Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (6 steps)

NEW: teachers get free access to ad-free/AI-free mode



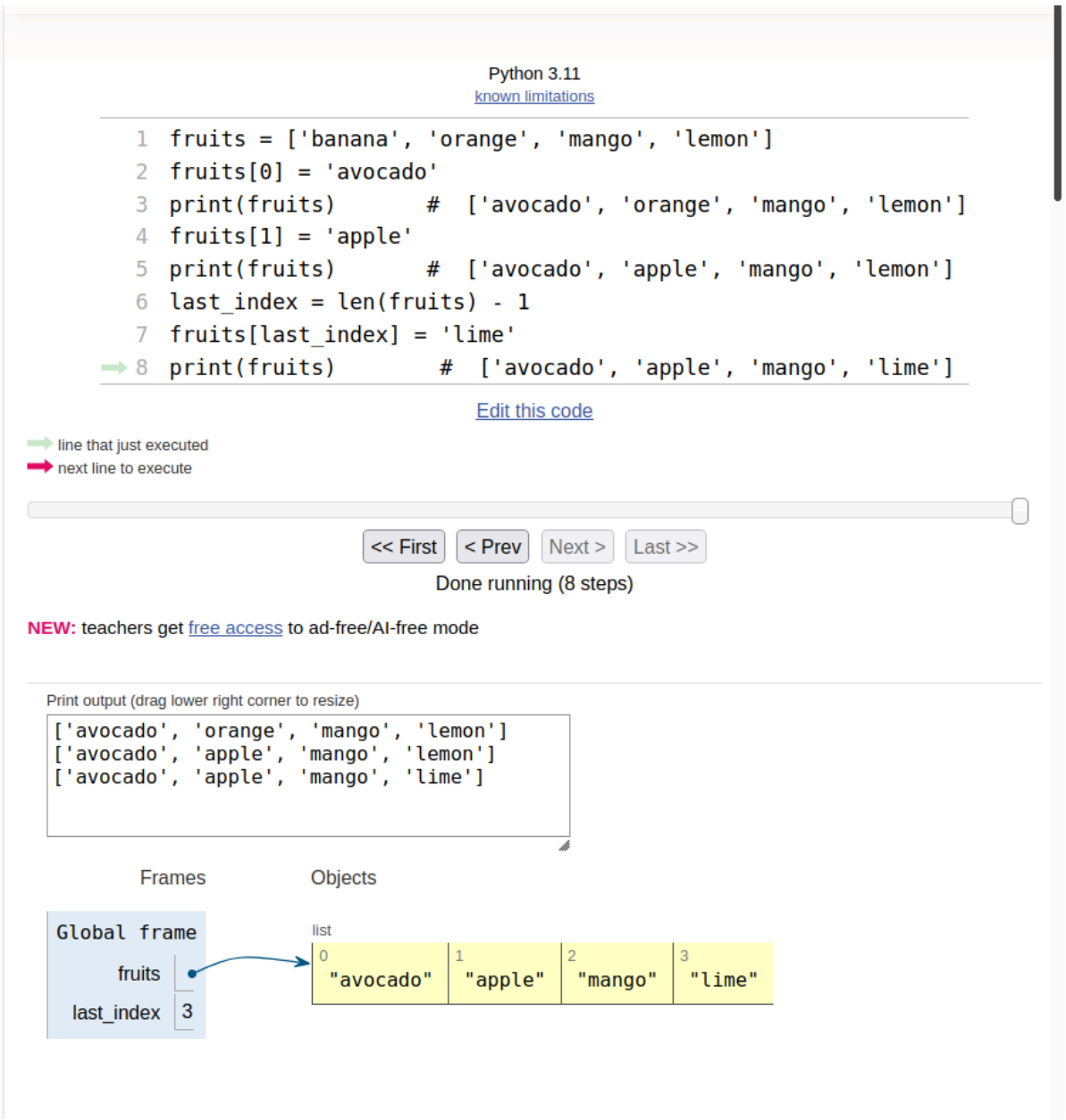
- Indexación negativa: Podemos especificar un rango de índices negativos especificando el inicio, final y paso, el valor de retorno será una nueva lista.

fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[-4:] # it returns all the fruits
orange_and_mango = fruits[-3:-1] # it does not include the last index,['orange', 'mango']
orange_mango_lemon = fruits[-3:] # this will give starting from -3 to the end,['orange', 'mango', 'lemon']
reverse_fruits = fruits[::-1] # a negative step will take the list in reverse order,['lemon', 'mango', 'orange', 'banana']

Modificar las listas

La lista es una colección ordenada mutable o modificable de artículos. Modifiquemos la lista de frutas.

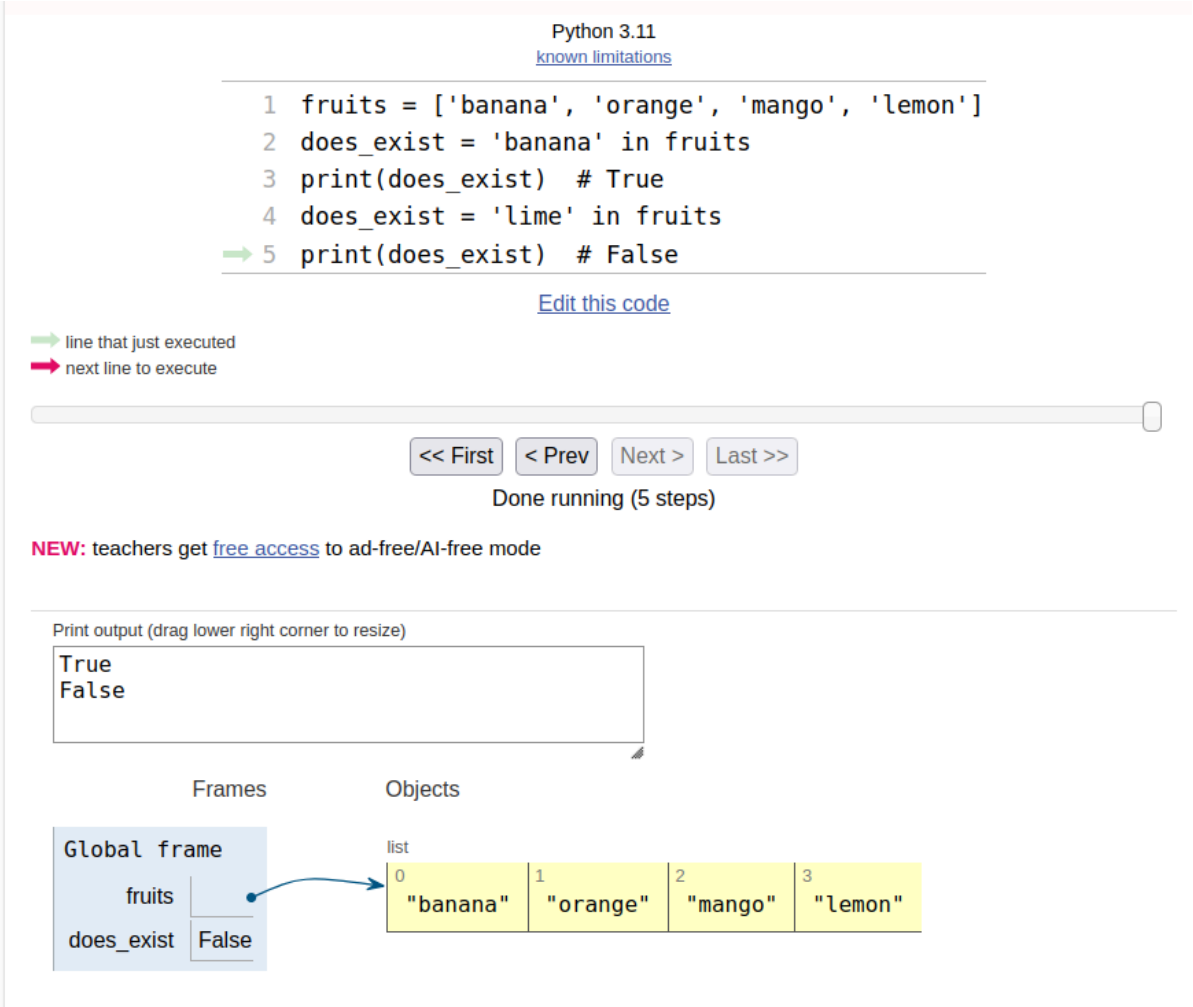
```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits[0] = 'avocado'
print(fruits)      #  ['avocado', 'orange', 'mango', 'lemon']
fruits[1] = 'apple'
print(fruits)      #  ['avocado', 'apple', 'mango', 'lemon']
last_index = len(fruits) - 1
fruits[last_index] = 'lime'
print(fruits)      #  ['avocado', 'apple', 'mango', 'lime']
```



Comprobar los artículos en una lista

Comprobar un elemento si es miembro de una lista que se utiliza *en* el operador. Vea el ejemplo a continuación.

```
fruits = ['banana', 'orange', 'mango', 'lemon']
does_exist = 'banana' in fruits
print(does_exist)  # True
does_exist = 'lime' in fruits
print(does_exist)  # False
```



Añadir artículos a una lista

Para añadir un elemento al final de una lista existente utilizamos el método *apend*().

```
# syntax
lst = list()
lst.append(item)

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.append('apple')
print(fruits)          # ['banana', 'orange', 'mango', 'lemon', 'apple']
fruits.append('lime')   # ['banana', 'orange', 'mango', 'lemon', 'apple', 'lime']
print(fruits)
```

```
1 lst = list()
2 item = 'Sandia'
3 lst.append(item)
4 fruits = ['banana', 'orange', 'mango', 'lemon']
5 fruits.append('apple')
6 print(fruits)          # ['banana', 'orange', 'mango', 'lemon', 'apple']
7 fruits.append('lime')   # ['banana', 'orange', 'mango', 'lemon', 'apple',
8 print(fruits)
```

Edit this code

line that just executed
next line to execute

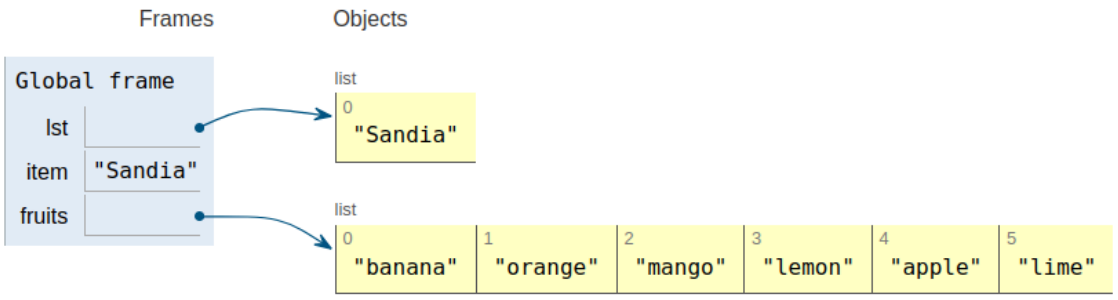
<< First | < Prev | Next > | Last >>

Done running (8 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'orange', 'mango', 'lemon', 'appl
['banana', 'orange', 'mango', 'lemon', 'appl
```



Insertar elementos en una lista

Podemos utilizar el método *insert()* para insertar un único elemento en un índice especificado en una lista. Tenga en cuenta que otros elementos se desplazan hacia la derecha. Los métodos *insert()* toman dos argumentos:índice y un elemento para insertar.

```
# syntax
lst = ['item1', 'item2']
lst.insert(index, item)

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.insert(2, 'apple') # insert apple between orange and mango
print(fruits)          # ['banana', 'orange', 'apple', 'mango', 'lemon']
fruits.insert(3, 'lime')   # ['banana', 'orange', 'apple', 'lime', 'mango', 'lemon']
print(fruits)
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.insert(2, 'apple') # insert apple between orange and mango
3 print(fruits)           # ['banana', 'orange', 'apple', 'mango', 'lemon']
4 fruits.insert(3, 'lime')   # ['banana', 'orange', 'apple', 'lime', 'mango
5 print(fruits)
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'orange', 'apple', 'mango', 'lemo
['banana', 'orange', 'apple', 'lime', 'mango
```

Frames | Objects

Global frame
fruits

list

0	1	2	3	4	5
"banana"	"orange"	"apple"	"lime"	"mango"	"lemon"

Eliminar artículos de una lista

El método de eliminación elimina un elemento especificado de una lista

# syntax
lst = ['item1', 'item2']
lst.remove(item)

fruits = ['banana', 'orange', 'mango', 'lemon', 'banana']
fruits.remove('banana')
print(fruits) # ['orange', 'mango', 'lemon', 'banana'] - this method removes the first occurrence of the item in the list
fruits.remove('lemon')
print(fruits) # ['orange', 'mango', 'banana']

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'banana']
2 fruits.remove('banana')
3 print(fruits)  # ['orange', 'mango', 'lemon', 'banana'] - this method rem
4 fruits.remove('lemon')
5 print(fruits)  # ['orange', 'mango', 'banana']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'banana']
['orange', 'mango', 'banana']
```

Frames | Objects

Global frame
fruits

list
0 "orange" | 1 "mango" | 2 "banana"

Eliminar artículos usando Pop

El método *pop()* elimina el índice especificado (o el último elemento si no se especifica el índice):

```
# syntax
lst = ['item1', 'item2']
lst.pop()      # last item
lst.pop(index)

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.pop()
print(fruits)      # ['banana', 'orange', 'mango']

fruits.pop(0)
print(fruits)      # ['orange', 'mango']
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.pop()
3 print(fruits)       # ['banana', 'orange', 'mango']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 2 of 3

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame
fruits

list

0	1	2	3
"banana"	"orange"	"mango"	"lemon"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.pop()
3 print(fruits)       # ['banana', 'orange', 'mango']
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (3 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'orange', 'mango']
```

Frames | Objects

Global frame
fruits

list

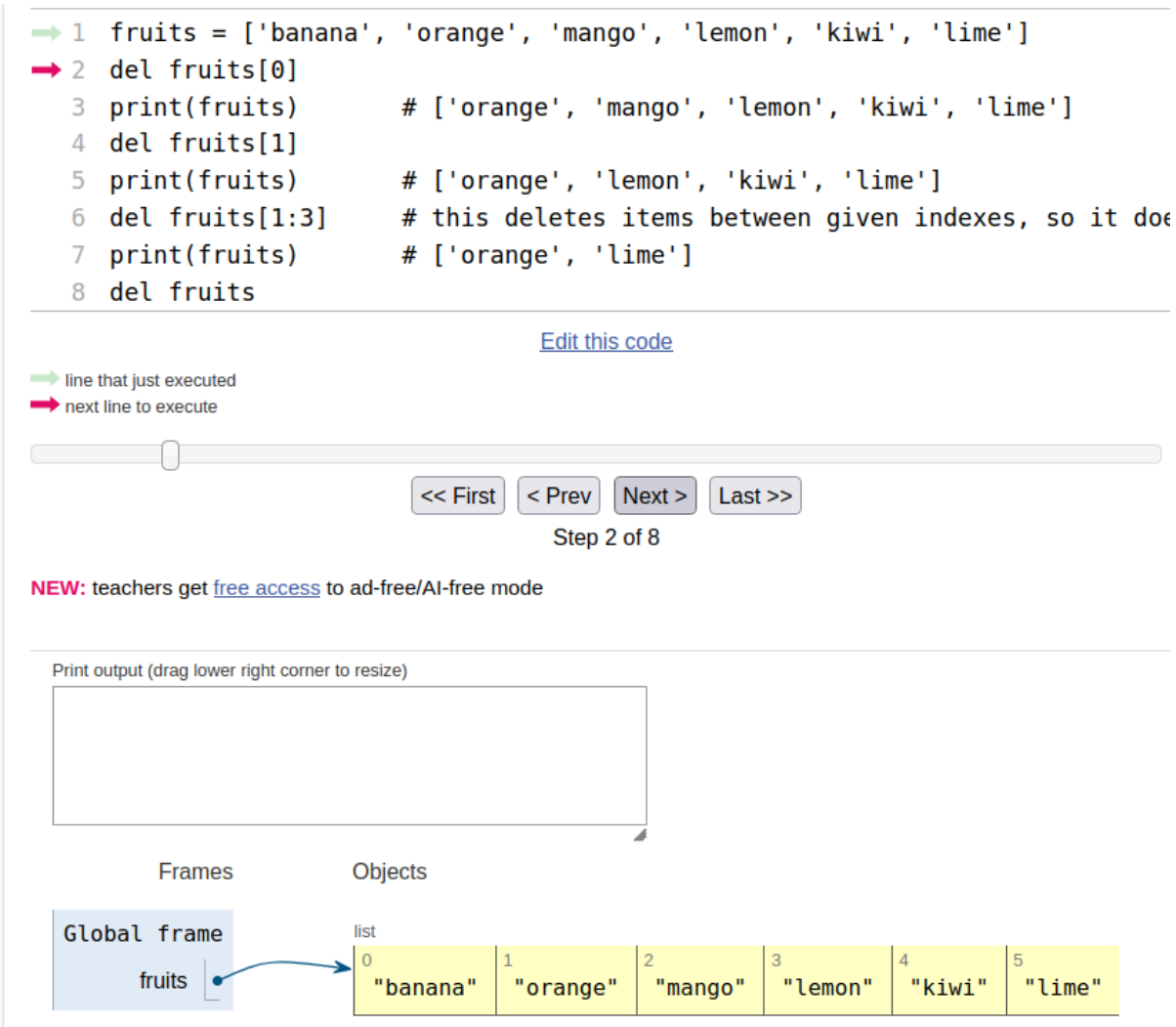
0	1	2
"banana"	"orange"	"mango"

Eliminar artículos usando Del

La palabra clave *del* elimina el índice especificado y también se puede utilizar para eliminar elementos dentro del rango de índice. También puede eliminar la lista por completo

```
# syntax
lst = ['item1', 'item2']
del lst[index] # only a single item
del lst        # to delete the list completely

fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
del fruits[0]
print(fruits)      # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
del fruits[1]
print(fruits)      # ['orange', 'lemon', 'kiwi', 'lime']
del fruits[1:3]    # this deletes items between given indexes, so it does not delete the item
with index 3!
print(fruits)      # ['orange', 'lime']
del fruits
print(fruits)      # This should give: NameError: name 'fruits' is not defined
```



```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 3 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

Frames | Objects

Global frame

fruits

list

0	1	2	3	4
"orange"	"mango"	"lemon"	"kiwi"	"lime"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Step 5 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
```

Frames | Objects

Global frame

fruits

list

0	1	2	3
"orange"	"lemon"	"kiwi"	"lime"

```
1 fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
2 del fruits[0]
3 print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
4 del fruits[1]
5 print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
6 del fruits[1:3]     # this deletes items between given indexes, so it doe
7 print(fruits)       # ['orange', 'lime']
8 del fruits
```

Edit this code

line that just executed
next line to execute

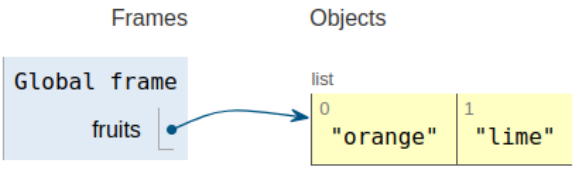
<< First | < Prev | Next > | Last >>

Step 7 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
['orange', 'lemon', 'kiwi', 'lime']
```



```
fruits = ['banana', 'orange', 'mango', 'lemon', 'kiwi', 'lime']
del fruits[0]
print(fruits)       # ['orange', 'mango', 'lemon', 'kiwi', 'lime']
del fruits[1]
print(fruits)       # ['orange', 'lemon', 'kiwi', 'lime']
del fruits[1:3]     # this deletes items between given indexes, so it doe
print(fruits)       # ['orange', 'lime']
del fruits
```

Edit this code

line that just executed
next line to execute

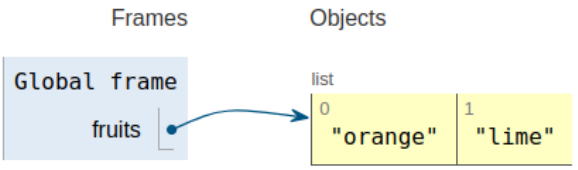
<< First | < Prev | Next > | Last >>

Step 8 of 8

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['orange', 'mango', 'lemon', 'kiwi', 'lime']
['orange', 'lemon', 'kiwi', 'lime']
['orange', 'lime']
```



Lista de compensación Artículos

El método *clear()* vacía la lista:

```
# syntax
lst = ['item1', 'item2']
lst.clear()

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.clear()
print(fruits)       # []
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.clear()
3 print(fruits)       # []
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (3 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
[]
```

Frames | Objects

Global frame
fruits → empty list

Copiar una lista

Es posible copiar una lista reasignándola a una nueva variable de la siguiente manera: list2 = list1. Ahora, list2 es una referencia de list1, cualquier cambio que hagamos en list2 también modificará el original, list1. Pero hay muchos casos en los que no nos gusta modificar el original en lugar de que nos guste tener una copia diferente. Una forma de evitar el problema anterior es usar *copy()*.

```
# syntax
lst = ['item1', 'item2']
lst_copy = lst.copy()

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits_copy = fruits.copy()
print(fruits_copy)      # ['banana', 'orange', 'mango', 'lemon']
```

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits_copy = fruits.copy()
3 print(fruits_copy)       # ['banana', 'orange', 'mango', 'lemon']
```

Edit this code

line that just executed
next line to execute

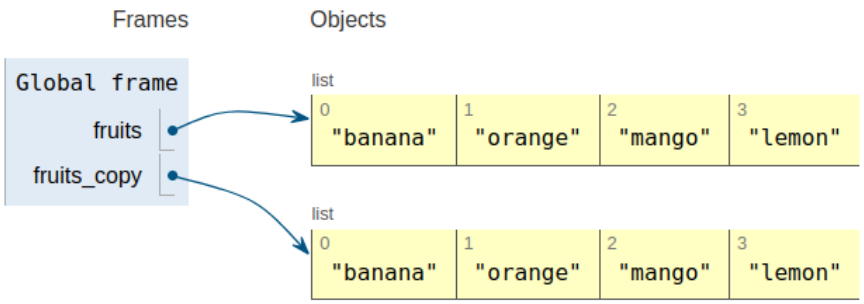
<< First | < Prev | Next > | Last >>

Done running (3 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'orange', 'mango', 'lemon']
```



Listas de unión

Hay varias maneras de unirse, o concatenar, dos o más listas en Python.

- Plus Operador (+)

# syntax
list3 = list1 + list2

positive_numbers = [1, 2, 3, 4, 5]
zero = [0]
negative_numbers = [-5,-4,-3,-2,-1]
integers = negative_numbers + zero + positive_numbers
print(integers) # [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]

fruits = ['banana', 'orange', 'mango', 'lemon']
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
fruits_and_vegetables = fruits + vegetables
print(fruits_and_vegetables) # ['banana', 'orange', 'mango', 'lemon', 'Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']

Python 3.11
known limitations

```
1 positive_numbers = [1, 2, 3, 4, 5]
2 zero = [0]
3 negative_numbers = [-5,-4,-3,-2,-1]
4 integers = negative_numbers + zero + positive_numbers
5 print(integers) # [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```

Edit this code

line that just executed
next line to execute

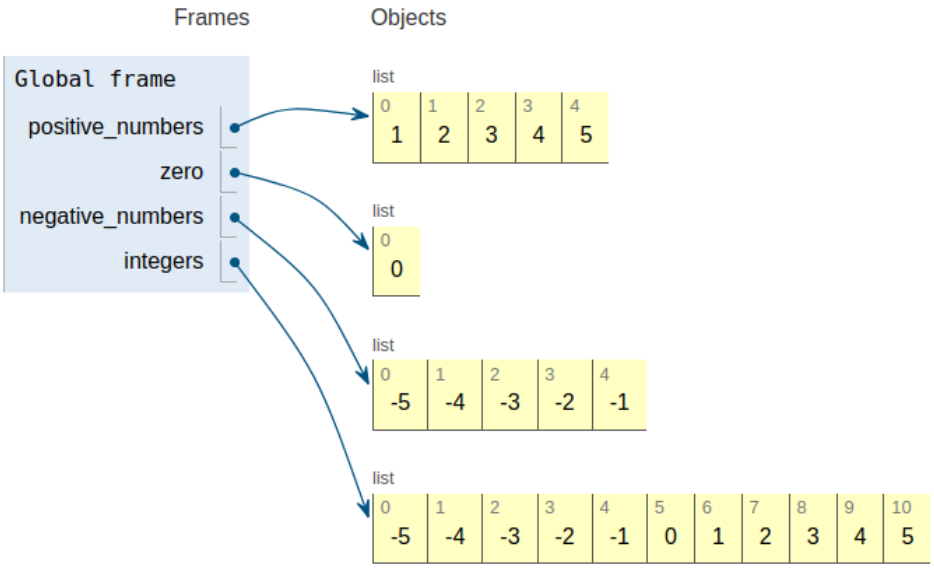
<< First | < Prev | Next > | Last >>

Done running (5 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
```



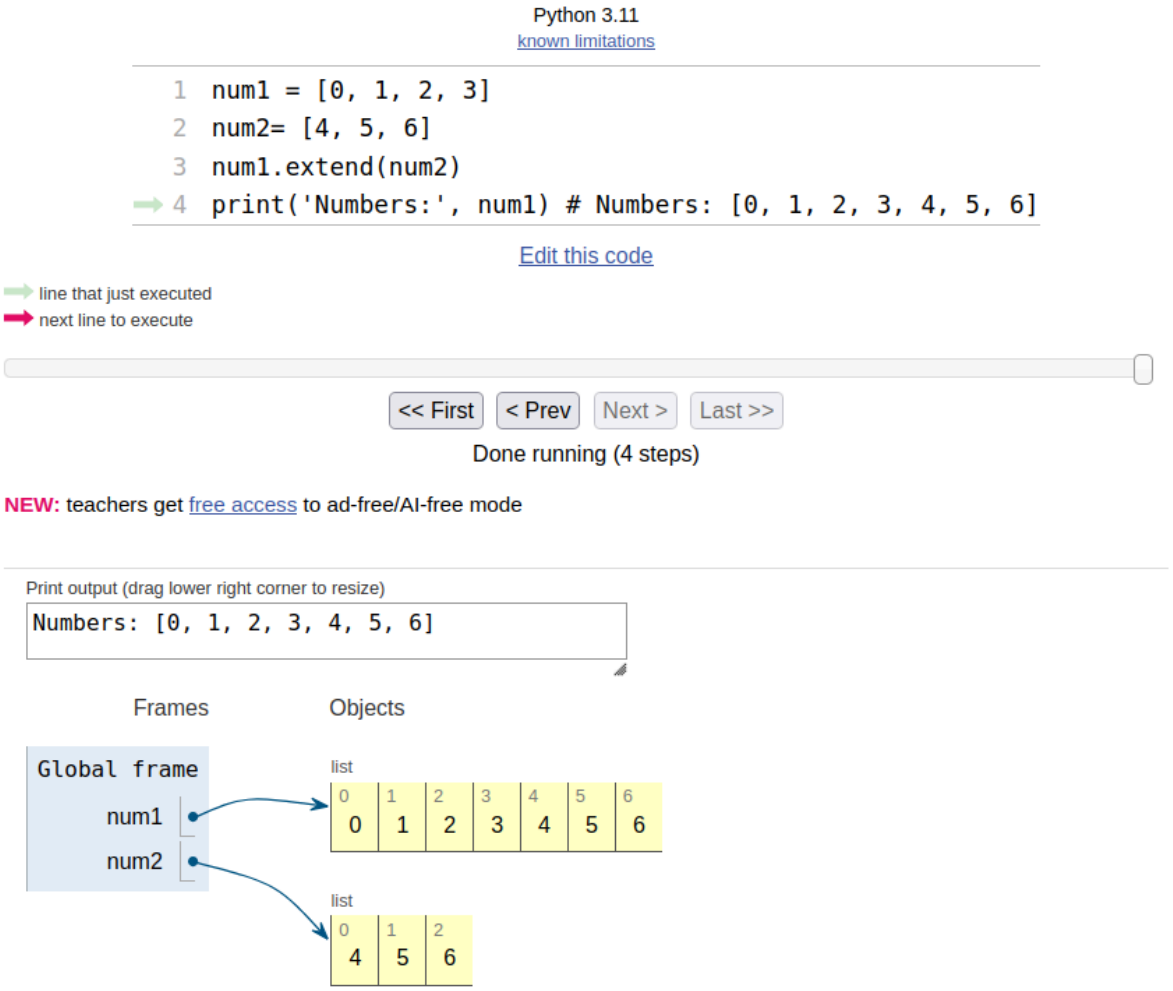
- Unir usando el método extend() El método *extend()* permite añadir la lista en una lista. Vea el ejemplo a continuación.

```
# syntax
list1 = ['item1', 'item2']
list2 = ['item3', 'item4', 'item5']
list1.extend(list2) # ['item1', 'item2', 'item3', 'item4', 'item5']

num1 = [0, 1, 2, 3]
num2= [4, 5, 6]
num1.extend(num2)
print('Numbers:', num1) # Numbers: [0, 1, 2, 3, 4, 5, 6]
```

```
negative_numbers = [-5,-4,-3,-2,-1]
positive_numbers = [1, 2, 3,4,5]
zero = [0]

negative_numbers.extend(zero)
negative_numbers.extend(positive_numbers)
print('Integers:', negative_numbers) # Integers: [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]
fruits = ['banana', 'orange', 'mango', 'lemon']
vegetables = ['Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
fruits.extend(vegetables)
print('Fruits and vegetables:', fruits ) # Fruits and vegetables: ['banana', 'orange', 'mango',
'lemon', 'Tomato', 'Potato', 'Cabbage', 'Onion', 'Carrot']
```

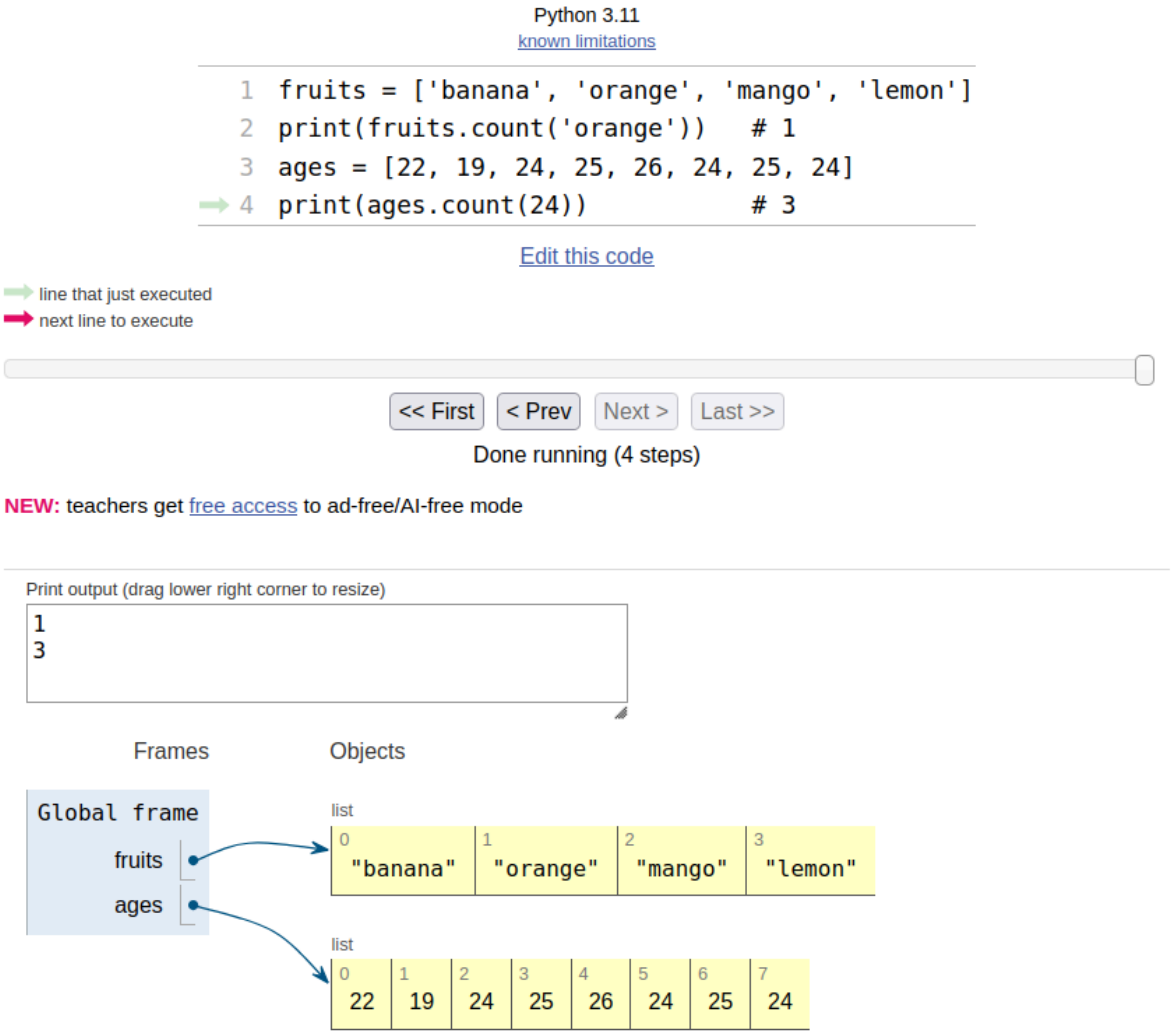


Contar artículos en una lista

El método *count()* devuelve el número de veces que un elemento aparece en una lista:

```
# syntax
lst = ['item1', 'item2']
lst.count(item)
```

```
fruits = ['banana', 'orange', 'mango', 'lemon']
print(fruits.count('orange'))   # 1
ages = [22, 19, 24, 25, 26, 24, 25, 24]
print(ages.count(24))          # 3
```



Índice de búsqueda de un artículo

El método *index()* devuelve el índice de un elemento de la lista:

```
# syntax
lst = ['item1', 'item2']
lst.index(item)

fruits = ['banana', 'orange', 'mango', 'lemon']
print(fruits.index('orange'))   # 1
ages = [22, 19, 24, 25, 26, 24, 25, 24]
print(ages.index(24))          # 2, the first occurrence
```

Python 3.11
known limitations

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 print(fruits.index('orange'))   # 1
3 ages = [22, 19, 24, 25, 26, 24, 25, 24]
4 print(ages.index(24))           # 2, the first occurrence
```

Edit this code

line that just executed
next line to execute

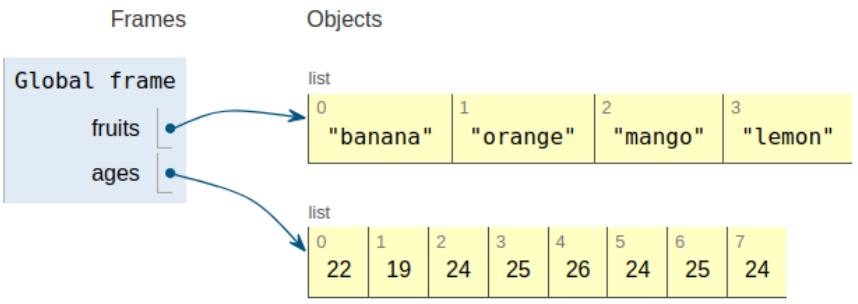
<< First | < Prev | Next > | Last >>

Done running (4 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
1
2
```



Revertir una lista

El método *reverse()* invierte el orden de una lista.

```
# syntax
lst = ['item1', 'item2']
lst.reverse()

fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.reverse()
print(fruits) # ['lemon', 'mango', 'orange', 'banana']
ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.reverse()
print(ages) # [24, 25, 24, 26, 25, 24, 19, 22]
```

Python 3.11
known limitations

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 fruits.reverse()
3 print(fruits) # ['lemon', 'mango', 'orange', 'banana']
4 ages = [22, 19, 24, 25, 26, 24, 25, 24]
5 ages.reverse()
6 print(ages) # [24, 25, 24, 26, 25, 24, 19, 22]
```

Edit this code

line that just executed
next line to execute

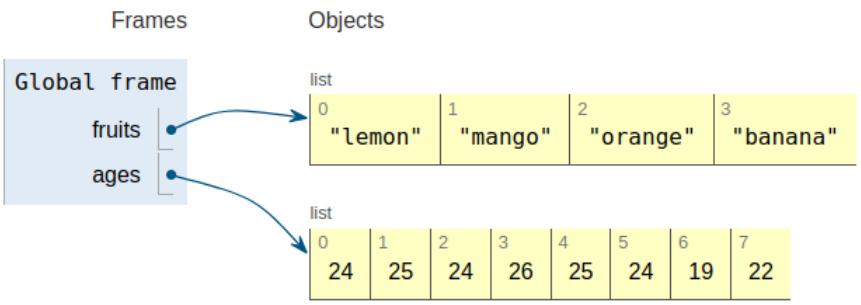
<< First | < Prev | Next > | Last >>

Done running (6 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['lemon', 'mango', 'orange', 'banana']
[24, 25, 24, 26, 25, 24, 19, 22]
```



Lista de clasificación de artículos

Para ordenar listas podemos utilizar el método *sort()* o las funciones integradas *sorted().* El método *sort()* reordena los elementos de la lista en orden ascendente y modifica la lista original. Si un argumento del método *sort()* inverso es igual a true, organizará la lista en orden descendente.

sort(): este método modifica la lista original

```
# syntax
lst = ['item1', 'item2']
lst.sort()              # ascending
lst.sort(reverse=True)    # descending
```

Ejemplo:

```
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits.sort()
print(fruits)          # sorted in alphabetical order, ['banana', 'lemon', 'mango', 'orange']
fruits.sort(reverse=True)
print(fruits) # ['orange', 'mango', 'lemon', 'banana']
ages = [22, 19, 24, 25, 26, 24, 25, 24]
ages.sort()
print(ages) #  [19, 22, 24, 24, 24, 25, 25, 26]

ages.sort(reverse=True)
print(ages) #  [26, 25, 25, 24, 24, 24, 22, 19]
```

```
1  fruits = ['banana', 'orange', 'mango', 'lemon']
2  fruits.sort()
3  print(fruits)            # sorted in alphabetical order, ['banana', 'le
4  fruits.sort(reverse=True)
5  print(fruits) # ['orange', 'mango', 'lemon', 'banana']
6  ages = [22, 19, 24, 25, 26, 24, 25, 24]
7  ages.sort()
8  print(ages) #  [19, 22, 24, 24, 24, 25, 25, 26]
9
10 ages.sort(reverse=True)
```

Edit this code

line that just executed
next line to execute

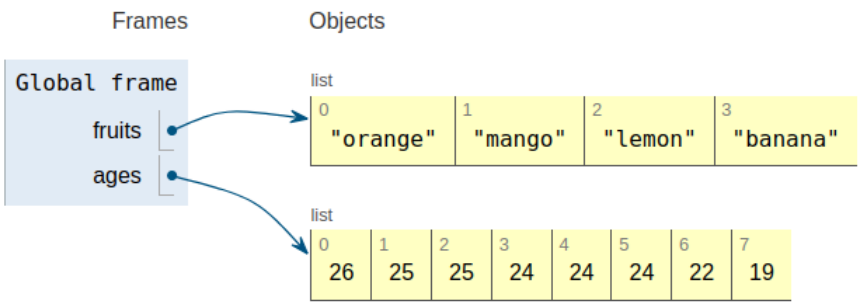
<< First | < Prev | Next > | Last >>

Done running (10 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'lemon', 'mango', 'orange']
['orange', 'mango', 'lemon', 'banana']
[19, 22, 24, 24, 24, 25, 25, 26]
[26, 25, 25, 24, 24, 24, 22, 19]
```



sorted(): devuelve la lista ordenada sin modificar la lista original

Ejemplo:

```
fruits = ['banana', 'orange', 'mango', 'lemon']
print(sorted(fruits))   # ['banana', 'lemon', 'mango', 'orange']
# Reverse order
fruits = ['banana', 'orange', 'mango', 'lemon']
fruits = sorted(fruits,reverse=True)
```

- print(fruits) # ['orange', 'mango', 'lemon', 'banana']

Python 3.11

known limitations

```
1 fruits = ['banana', 'orange', 'mango', 'lemon']
2 print(sorted(fruits))   # ['banana', 'lemon', 'mango', 'orange']
3 # Reverse order
4 fruits = ['banana', 'orange', 'mango', 'lemon']
5 fruits = sorted(fruits,reverse=True)
```

Edit this code

line that just executed

next line to execute

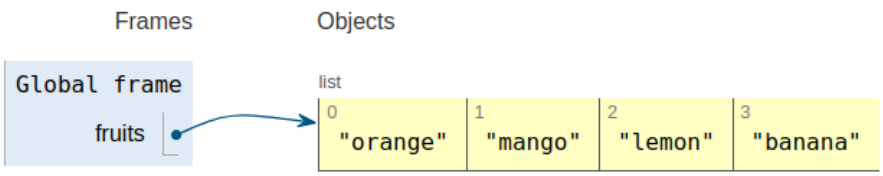
<< First | < Prev | Next > | Last >>

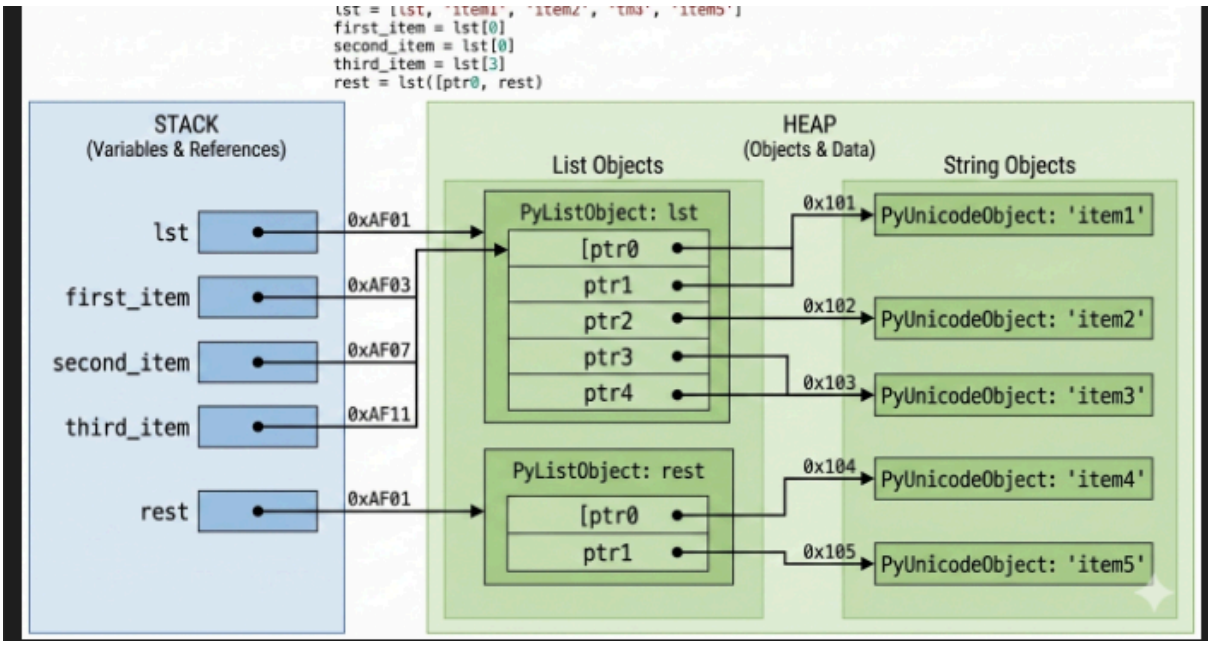
Done running (4 steps)

NEW: teachers get free access to ad-free/AI-free mode

Print output (drag lower right corner to resize)

```
['banana', 'lemon', 'mango', 'orange']
```




lst = [lst, 'item1', 'item2', 'tm4', 'item5']
first_item = lst[0]
second_item = lst[0]
third_item = lst[3]
rest = lst([ptr0, rest)
STACK
(Variables & References)
lst
first_item
second_item
third_item
rest
0xAF01
0xAF03
0xAF07
0xAF11
0xAF01
HEAP
(Objects & Data)
List Objects
String Objects
PyListObject: lst
[ptr0
ptr1
ptr2
ptr3
ptr4
PyListObject: rest
[ptr0
ptr1
0x101
0x102
0x103
0x104
0x105
PyUnicodeObject: 'item1'
PyUnicodeObject: 'item2'
PyUnicodeObject: 'item3'
PyUnicodeObject: 'item4'
PyUnicodeObject: 'item5'